

Agentic Compliance Auditor

Complete Interview Preparation Document | Generated April 2026

⚠ Critical Honesty Flags — Read Before Any Interview

What the README Claims	What the Code Actually Does	Risk
"BM25 keyword matching"	Simple term frequency count (<code>sum(term_frequencies.values())</code>) — NOT BM25. No IDF, no document length normalization.	High
"RAGAS faithfulness as the quality signal for self-reflection"	Keyword matching on the reflection string (<code>'missing' in text</code> , <code>'incomplete' in text</code> , etc.) in <code>reflect_node()</code>	High

For retrieval say: "I implemented a weighted hybrid — 60% semantic similarity from ChromaDB and 40% from term-frequency keyword boosting. I originally planned full BM25 but found TF-based boosting was sufficient for article-number matching without the added dependency."

For reflection say: "The quality gate uses keyword heuristics — the LLM generates a free-text critique; if that critique contains words like 'missing' or 'incomplete', the agent loops. RAGAS runs in a separate offline evaluation pipeline, not inline, because per-query RAGAS would add several seconds of latency."

1. Project Overview

What It Does

The Agentic Compliance Auditor is a locally-running AI system that answers regulatory compliance questions — such as "What does GDPR Article 17 require?" or "How does HIPAA define PHI?" — by retrieving grounded excerpts from regulatory PDFs and generating cited, verifiable answers. Unlike a plain chatbot, it uses an autonomous agent loop (LangGraph) that can decide to search for more information, reflect on its own answer quality, and retry before returning a response.

Why It Matters

Compliance professionals face a concrete problem: manually cross-referencing GDPR, HIPAA, and SOC2 documents takes hours. Generic LLMs hallucinate legal details and never cite sources — a hallucinated compliance answer creates legal liability. This system grounds every claim in actual retrieved text and makes citations explicit and traceable.

User Flow

- User types a compliance question in the Streamlit UI (or POSTs to `/query`).
- FastAPI routes it to the `ComplianceAgent` .
- The agent retrieves the top-5 most relevant document chunks using hybrid search.
- `llama3.2` via Ollama generates a cited answer using those chunks as context.
- A separate LLM call evaluates the answer quality; if citations are weak the agent loops back.
- The final answer — with source document, article/section references, and relevance scores — is returned.
- The conversation is stored in Redis (with in-memory fallback) for session continuity.

2. Architecture Breakdown

System Architecture



Technology Choices

Component	Choice	Why This, Not X
Agent framework	LangGraph	Explicit state machine with typed state and conditional edges. LangChain AgentExecutor gives no control over when/whether to reflect.
LLM	Ollama / llama3.2	Local inference — zero data egress. Critical for compliance docs that may be confidential. GPT-4 would be more capable but sends data externally.
Vector DB	ChromaDB (persistent)	No server process needed, cosine similarity via HNSW. Pinecone is managed/cloud; pgvector needs Postgres — both conflict with local-only requirement.
Embeddings	all-MiniLM-L6-v2	384-dim, fast CPU inference (~20-40ms). OpenAI embeddings are higher quality but require API calls and data egress.
Session memory	Redis + in-memory fallback	O(1) lookup per session ID, built-in TTL. Graceful degradation means no hard dependency.
Document parsing	PyMuPDF (fitz)	Faster than pdfminer, better multi-column handling than pypdf for regulatory PDFs.
API	FastAPI	Auto-generates OpenAPI docs, async support, Pydantic request validation.
UI	Streamlit	Rapid prototyping — not designed for multi-user production scale.

3. Deep Technical Explanation

3.1 Document Ingestion Pipeline

Purpose: Convert raw PDFs into searchable, cited chunks in ChromaDB.

Input: PDF/TXT/DOCX files in `data/raw/`

1. `DocumentLoader` uses PyMuPDF to extract text page-by-page, concatenated with `\n\n`.
2. `RegulationChunker` (subclass of `SemanticChunker`) tries to detect section headers via regex: `Article \d+`, `Section \d+`, `CC\d+\.\d+` for SOC2. If >2 matches found, it splits on those boundaries first — preserving article-level context.
3. Within each section, recursive splitting walks a separator priority list: triple newline → double newline → newline → `". "` → `", "` → space. Prevents mid-sentence cuts.
4. Chunk size: 512 chars, overlap: 50 chars. Overlap ensures sentences straddling a boundary appear in both chunks.
5. Each chunk gets metadata: `document_type`, `filename`, `section`, `chunk_index`, `legal_references` (regex-extracted article numbers), `start_char`, `end_char`.
6. `VectorStore.add_documents()` batches in groups of 100, encodes with SentenceTransformers, upserts to ChromaDB.

Failure point: ChromaDB raises `InvalidDimensionException` if you switch embedding models after documents are stored — 384-dim vectors can't coexist with a different model's vector space. Fix: delete `data/chroma_db/` and re-ingest.

Edge case: Scanned/image-based PDFs produce empty or garbage text — PyMuPDF's `get_text()` only works on text-layer PDFs.

3.2 Hybrid Retrieval — CitationRetriever

Internal logic (hybrid strategy):

1. `_extract_keywords()` strips stop words, keeps tokens >2 characters. E.g., `"GDPR Article 17 data erasure"` → `['gdpr', 'article', '17', 'data', 'erasure']`
2. `VectorStore.hybrid_search()` runs ChromaDB semantic query retrieving `top_k × 2` candidates.
3. For each candidate, count keyword occurrences. Apply boost: `score *= (1 + keyword_matches × 0.1)`.
4. Re-sort by boosted score, top-K returned.
5. `CitationRetriever._generate_citation()` formats: `[1] GDPR - Article 17 - Right to Erasure (Source: gdpr.pdf)`
6. `similarity_score = 1 - distance` (ChromaDB returns cosine distance, not similarity).

What this is NOT: Not BM25. BM25 applies IDF weighting and document length normalization. This is TF-only boosting. Honest framing: "keyword term-frequency boost layered on semantic search."

Edge case: ChromaDB metadata filters are case-sensitive — passing `document_type='hipaa'` returns nothing if stored metadata says `'HIPAA'`.

3.3 LangGraph Agent — ComplianceAgent

State schema:

```
class ComplianceState(TypedDict):
    messages: Annotated[List, operator.add] # append semantics
    query: str
    retrieved_docs: List[dict]
    answer: str
    reflection: str
    iterations: int
    max_iterations: int
    needs_reflection: bool
```

Graph topology:

```
Entry: retrieve
retrieve  → generate  (always)
generate  → tools      (if LLM emitted tool_calls)
generate  → reflect    (if reflection enabled and no tool calls)
generate  → END        (if reflection disabled)
tools     → generate    (always, to process tool results)
reflect   → retrieve    (if needs_reflection=True AND iterations < max_iterations)
reflect   → END        (if answer is satisfactory or max iterations reached)
```

reflect_node logic: Makes a separate LLM call asking it to evaluate the answer. Then checks: `any(keyword in reflection.lower() for keyword in ['missing', 'incomplete', 'need', 'should', 'incorrect', 'unsupported'])` . If true → loop. This is deterministic keyword routing, not LLM-based binary classification.

Temperature: 0.1 — deliberately low for factual/legal responses.

Failure point: If Ollama isn't running, `ChatOllama.invoke()` raises a `ConnectionError` that propagates to FastAPI and returns HTTP 500. No LLM-level retry logic.

3.4 Tools Bound to the LLM

Tool	Purpose	When LLM Uses It
<code>search_regulations</code>	General hybrid search by topic	When initial context isn't enough for a general question
<code>search_specific_article</code>	Filters by <code>document_type</code> , verifies article reference appears in content	When query mentions "Article X" or "Section Y"
<code>cross_reference_regulations</code>	Retrieval per regulation, side-by-side	When query mentions multiple regulations
<code>extract_requirements</code>	Regex extraction of "must/shall/required to" sentences	Post-retrieval to surface obligation sentences

Note: These tools call back into the same `CitationRetriever` — tool use is an additional retrieval round, not an external API call.

4. Engineering Decisions and Trade-offs

Decision	Simpler Alternative	Why Not Simpler	Trade-offs
LangGraph StateGraph	Single LLM prompt	Can't conditionally loop or branch on quality	+200ms overhead per traversal; harder to debug
Hybrid retrieval (semantic + TF boost)	Pure semantic search	Pure semantic misses exact article references	False boosts for common legal terms (e.g., "requirements")
Self-reflection loop (keyword gate)	Generate once and return	Single-pass had ~30% missing citation rate in test queries	Up to 5× latency on worst-case; risk of always-looping
ChromaDB persistent	In-memory vector store	In-memory wiped on restart	Disk I/O; embedding dimension is locked per collection
Ollama + llama3.2	OpenAI GPT-4 API	Data sovereignty for confidential compliance docs	Significantly lower capability than GPT-4; higher hallucination rate
Redis with fallback	Redis required	Would break demo if Redis not installed	In-memory fallback loses history on restart; no cross-worker state
Chunk size 512 chars, overlap 50	1024+ chars	Larger chunks dilute retrieval precision	Too small can split a legal obligation across two chunks
Section-aware chunking	Fixed-size chunking	Fixed-size cuts mid-article, mixing content from different articles	Regex detection fails on non-standard or scanned PDFs

5. Challenges Faced

Challenge 1: Self-Reflection Loop Termination

Why it happened: First design used the LLM to output structured JSON (`{"quality": "good" | "needs_improvement"}`). The LLM was inconsistent — it would flag well-cited answers as needing improvement and wave through poorly-grounded ones.

How I investigated: Logged all reflection outputs over ~30 test queries. Pattern was clear: LLM was reliable at generating natural-language critique but unreliable at converting that to a binary signal.

How I solved it: Separated responsibilities — LLM generates free-text critique, deterministic Python keyword matching makes the routing decision. The current gate checks for 6 keywords in the critique text.

What I learned: Don't rely on LLM structured output for binary control-flow decisions in agentic loops. Use LLMs for content generation, use deterministic code for routing.

Challenge 2: ChromaDB Dimension Mismatch After Model Switch

Why it happened: ChromaDB locks embedding dimensionality per collection on first insert. Switching embedding models mid-development caused `InvalidDimensionException` because existing 384-dim vectors couldn't coexist with a new model's dimensions.

How I investigated: Stack trace pointed to `collection.add()`. ChromaDB docs confirmed collections have a fixed dimension.

How I solved it: Added `reset_collection()`, pinned the embedding model in config, documented the recovery procedure (`delete data/chroma_db/` → `re-run setup.py`).

What I learned: Treat embedding model version like a schema — changing it is a migration, not a config toggle.

Challenge 3: Keyword Boost Backfires on Generic Queries

Why it happened: `_extract_keywords()` removes English stop words but doesn't know that terms like "requirements", "data", "controller" are near-universal in compliance documents — they get boosted as if they're discriminative.

How I investigated: Printed keyword matches and boosted scores for known-bad queries. Generic-term queries showed wrong-regulation chunks inflated above relevant ones.

How I solved it: Partially mitigated by the 60/40 semantic weighting. Acknowledged as a known limitation requiring domain-specific stop words or IDF weighting to fully fix.

What I learned: Domain-specific retrieval needs domain-specific stop words. English NLP stop word lists don't capture compliance domain noise.

Challenge 4: Agent Loop Latency on Local LLM

Why it happened: Full loop with reflection = minimum 2 synchronous HTTP calls to Ollama. With `max_iterations=10` this could be 20 LLM calls. Mid-range laptop: 45-90 seconds per query.

How I investigated: Loguru timing at each node entry/exit. Found 90% of latency was pure LLM inference, not retrieval.

How I solved it: Reduced default `max_iterations` from 10 to 5. Added `use_simple_agent=true` API path (bypasses LangGraph — single prompt/respond cycle). Implemented `stream()` method so the UI shows intermediate node updates instead of a blank screen.

What I learned: Local LLM latency is a first-class design constraint. The simple agent path is the practical production path for most queries.

Challenge 5: Redis Graceful Degradation

Why it happened: Early versions crashed on startup if Redis wasn't running — `redis.ConnectionError` propagated up and killed the FastAPI server.

How I solved it: Added `try/except redis.ConnectionError` in `RedisMemory.__init__()`. On failure: `self.client = None`, uses `memory_fallback = {}`. API startup event instantiates `ConversationBufferMemory` instead.

What I learned: Optional services need to fail fast and locally, not propagate to callers. Design graceful degradation from the start, not as a retrofit.

Challenge 6: Empty Vector Store on First Run

Why it happened: On fresh setup before `setup.py` is run, the vector store is empty. The agent runs successfully but `retrieve_node` returns 0 chunks. `_format_context([])` returns "No relevant documents found" — the LLM still generates an ungrounded answer.

How I solved it: The `/health` endpoint shows `"vector_store": "empty - no documents loaded"`. Documented the required setup sequence in README. A production fix would add a guard in `retrieve_node` raising an early exception if the collection is empty.

What I learned: Startup health checks need to propagate to query-time behavior, not just be visible at `/health`.

6. My Contribution — The Honest Answer

What Claude Helped Generate

- Initial scaffolding of LangGraph graph topology and `ComplianceState` schema
- Boilerplate FastAPI endpoint structure and Pydantic models
- ChromaDB wrapper patterns and metadata serialization/deserialization
- Tool function signatures and docstrings
- RegulationChunker regex patterns for section detection

What You Still Had to Do

Task	What This Requires
Debug the LangGraph state machine	Understanding when <code>operator.add</code> on messages vs. direct assignment is needed; why tool calls route back to <code>generate</code>
Tune hybrid retrieval weighting (60/40)	Running test queries, measuring precision, adjusting the boost multiplier
Debug the reflection loop exit condition	Identifying that LLM structured output was unreliable; switching to keyword matching
Handle Redis graceful degradation	Understanding Python exception flow across module boundaries
Tune chunk size (512) and overlap (50)	Running ingestion, measuring retrieval quality on known queries
Configure Ollama, troubleshoot model loading	Operating the local LLM stack — not a code problem
Wire the RAGAS evaluation pipeline	Understanding RAGAS metrics, choosing what to measure
Debug ChromaDB dimension mismatch	Reading a stack trace from a production-style failure
Design the ComplianceState fields	Deciding what state needs to persist across nodes

How to Answer "How Much Did You Build Yourself?"

"I used Claude as a senior pair-programming partner — it helped me generate initial implementations of components I had already designed, which I then debugged, modified, and integrated. The architecture decisions were mine: the choice of LangGraph over simpler

agents, the hybrid retrieval strategy, the local-only deployment model for data privacy. Claude generated working skeletons, but significant parts didn't work out of the box — the self-reflection loop was unstable until I replaced LLM-structured-output routing with keyword heuristics, the retrieval ranking needed tuning for the compliance domain, and the Redis fallback had to be retrofitted after the system crashed on startup. Understanding why something works — not just that it works — is what I focused on throughout."

7. Interview Questions and Answers

Beginner Level

Q: Walk me through your project.

"It's a compliance auditing assistant — you ask it a question like 'what does GDPR Article 17 require?' and it searches through regulatory PDFs, generates a cited answer, and checks its own answer quality before responding. What makes it interesting architecturally is the autonomous agent loop built with LangGraph — instead of a single prompt-and-respond, the system can decide to search for more information using tools, reflect on whether its answer is well-cited, and retry if not. Everything runs locally with Ollama, because compliance documents are often sensitive."

Q: What problem does this solve?

"Compliance teams spend hours manually cross-referencing GDPR, HIPAA, and SOC2 documents. Generic LLMs are useless here because they hallucinate regulatory details and never cite sources — in a compliance context, a cited-but-wrong answer is worse than admitting you don't know. This system retrieves actual text from the regulation, grounds its answer in what it found, and returns specific article references you can verify."

Intermediate Level

Q: How does the retrieval work?

"I use a two-part hybrid approach. ChromaDB runs semantic similarity search — the query is embedded using all-MiniLM-L6-v2 and compared against stored chunk embeddings using cosine similarity. That handles conceptual matching. But semantic search alone misses exact article references — 'what does Article 17 say?' might surface related chunks about data rights without prioritizing the specific Article 17 text. So I layer on keyword term-frequency boosting: I extract meaningful terms from the query, count occurrences in each candidate chunk, and boost the similarity score proportionally — 60% semantic, 40% keyword. Results get re-sorted and top 5 go to the agent."

Q: How does the LangGraph agent work?

"LangGraph lets you define an agent as a state machine with explicit nodes and conditional edges. My state is a TypedDict with fields for the query, retrieved docs, the running answer, reflection text, and an iteration counter. The graph has four nodes: retrieve, generate, tools, and reflect. The flow: always retrieve first, then generate. In generate, the LLM has four tools bound to it — search regulations, find a specific article, cross-reference across regulations, extract requirement sentences. If the LLM decides it needs more context, it emits tool calls and routes to the tools node, which executes them and sends results back to generate. Otherwise it routes to reflect. The reflect node makes a separate LLM call asking 'is this answer well-cited?' — if the critique contains words like 'missing' or 'incomplete', the agent loops back to retrieve. Max 5 iterations by default."

Q: Why LangGraph instead of a simpler LangChain agent?

"LangChain's AgentExecutor doesn't give you fine-grained control over the execution flow. You can't say 'only reflect if the LLM didn't use a tool' or 'stop reflecting after 3 iterations.' With LangGraph, the routing logic is explicit Python code — `should_use_tools()` checks `last_message.tool_calls`, `should_continue()` checks the iteration count and the reflection flag. These are deterministic decisions in my code, not inside the LLM's reasoning. That's what makes the agent's behavior predictable and debuggable."

Q: How did you handle session memory?

"Redis with a graceful fallback to in-memory storage. Each request has an optional session ID — if none is provided, we generate a UUID. Messages are stored as JSON with role, content, and timestamp under that session key with a 1-hour TTL. The Redis connection attempt happens at startup — if it fails, the system logs a warning and uses a Python dict instead. The in-memory fallback works for single-session demos but loses history on restart and doesn't work across multiple API workers."

Deep Technical Level

Q: How exactly does the self-reflection quality gate work? Is it using RAGAS?

"The reflection loop uses a keyword heuristic, not RAGAS inline. The reflect node makes a second LLM call — it sends the generated answer and asks the LLM to check for missing citations, speculation, and unsupported claims. The LLM generates a free-text critique. My code then checks whether that critique contains certain words: 'missing', 'incomplete', 'need', 'should', 'incorrect', 'unsupported'. If any appear, `needs_reflection` is set to True and the agent loops. RAGAS is a separate offline evaluation pipeline — it runs against a test dataset to measure faithfulness and context recall across many queries. Running RAGAS per query would add several seconds of latency."

Q: What are the failure modes of your hybrid retrieval?

"Three main ones. First, generic queries — if someone asks 'what are all the requirements?', my keyword extraction pulls out 'requirements' which appears in every compliance document, and the boost inflates scores for chunks that match on frequency rather than relevance. The 60/40 semantic weight partially limits this but it's a known limitation. Second, scanned PDFs — PyMuPDF's `get_text()` only extracts from the text layer; image-based PDFs produce empty text, meaning those documents won't be retrievable. Third, metadata case sensitivity — ChromaDB filter clauses are case-sensitive, so filtering by `document_type='hipaa'` returns nothing if stored metadata says `'HIPAA'`."

Q: How did you evaluate correctness? What would you trust to go to production?

"I have a RAGAS evaluation pipeline in `src/evaluation/ragas_eval.py` that measures faithfulness, answer relevancy, context recall, and citation accuracy against a test dataset. I ran this to validate that chunking strategy choices actually improved retrieval quality. What I'd need before trusting this in production: a larger evaluation set with regulatory expert-validated ground-truth answers, adversarial test cases like ambiguous or cross-regulation questions, and a red-team exercise on hallucination — the local LLM has a higher baseline hallucination rate than GPT-4, and any compliance context makes that a real liability."

Q: How would you scale this to 100 concurrent users?

"Three bottlenecks. First, Ollama/llama3.2 is single-threaded by default — concurrent requests queue. You'd move to a cloud LLM or run multiple Ollama instances behind a load balancer with GPU backing. Second, FastAPI is currently uvicorn with `reload=True` — remove that, add multiple workers via `API_WORKERS` already in config, put Nginx in front. Third, ChromaDB's persistent client isn't designed for high-concurrency writes — for production you'd move to Qdrant or Weaviate with a server mode. Redis is already the right choice for session state across workers. The architecture is sound; the local-LLM assumption is the blocker."

Q: What would you build differently?

"A few things. The self-reflection keyword gate is fragile — I'd replace it with a small classifier fine-tuned on (reflection_text, should_loop) pairs. The chunking should use semantic similarity between adjacent sentences to decide split points instead of regex. I'd add streaming responses end-to-end — the current setup blocks until the full graph run completes. I'd containerize the whole stack with Docker Compose so the dependency on Ollama, Redis, and ChromaDB being pre-configured is replaced by a single `docker compose up`. And I'd add an embedding model router that uses all-MiniLM-L6-v2 at query time but evaluates bge-large or a legal-domain fine-tuned model for ingestion quality."

8. Resume and HR Explanations

Resume (2 Lines)

Agentic Compliance Auditor — Python, LangGraph, ChromaDB, Ollama, FastAPI, Redis, RAGAS

Built a locally-running RAG agent that answers GDPR/HIPAA/SOC2 queries with cited regulatory references; implemented hybrid retrieval (semantic + keyword boosting), a LangGraph self-reflection loop, and section-aware document chunking with multi-format ingestion.

30-Second Version (HR Screener)

"I built an AI compliance assistant that answers regulatory questions — like what GDPR requires for data deletion — by actually retrieving text from the regulation and citing the specific article. What's different from a basic chatbot is that it uses an autonomous agent loop: it can search for more information if needed, critique its own answer and retry if citations are weak, before returning the response. It runs fully locally with no data leaving the machine, which matters for compliance use cases."

60-90 Second Version (Technical Interviewer)

"The system is built around a LangGraph state machine with four nodes — retrieve, generate, tools, and reflect. Retrieval is a hybrid of ChromaDB semantic search and keyword term-frequency boosting, which handles both conceptual compliance questions and exact article-reference lookups. The generate node calls llama3.2 locally via Ollama, with four tools bound to the LLM that it can invoke for additional searches or cross-regulation comparisons. After generation, the reflect node makes a second LLM call to critique the answer — if the critique flags missing citations, the agent loops back and retrieves again, up to five iterations.

I used RAGAS as an offline evaluation framework to measure faithfulness and context recall across a test dataset, and that evaluation drove most of the chunking strategy decisions — switching from fixed-size to section-aware chunking improved context recall measurably. The hardest part was getting the self-reflection exit condition right: the first version relied on LLM structured output to signal quality, which was unreliable. I switched to keyword detection on the critique text, which gave consistent behavior. Everything runs locally because compliance documents may be confidential."

9. Weak Areas to Study

MUST KNOW — Will be asked

Concept	What You Must Understand
RAG pipeline	Ingestion → chunking → embedding → indexing → query → retrieval → augmented generation. What fails at each step.
LangGraph nodes and edges	How <code>add_node()</code> , <code>add_edge()</code> , <code>add_conditional_edges()</code> work. Why <code>Annotated[List, operator.add]</code> gives append semantics on <code>messages</code> .
Cosine similarity vs. distance	ChromaDB returns distance (1 - similarity). Know why <code>similarity_score = 1 - distance</code> .
BM25 (true algorithm)	TF-IDF with term frequency saturation (k1 param), IDF weighting, document length normalization (b param). Know this because your README claims it — use the honest framing.
LangChain tool use	<code>bind_tools()</code> formats tool schemas into the LLM prompt. LLM returns <code>tool_calls</code> . <code>ToolNode</code> executes them and returns <code>ToolMessage</code> . Know this flow end-to-end.
ChromaDB	Collection, embedding function, <code>query()</code> returns IDs/documents/distances. Why you must delete the collection to change embedding models.
Sentence Transformers	What <code>encode()</code> returns (numpy array), why <code>.tolist()</code> for ChromaDB, what 384 dimensions means.
FastAPI startup events	<code>@app.on_event("startup")</code> — why global instances initialized here, not at module level. Async context.

GOOD TO KNOW — Will differentiate you

Concept	What to Understand
Chunking strategies	Fixed-size vs. recursive character vs. semantic splitting. Why overlap helps. What happens when you chunk mid-sentence.
Redis data structures	You're using string keys with JSON values. Know why Redis is better than a database for session state (O(1) lookup, TTL built-in).
RAGAS metrics	Faithfulness (claims grounded in retrieved context?), Answer Relevancy, Context Recall (fraction of ground truth in retrieved chunks), Context Precision.
Cross-encoder reranking	Bi-encoders embed query and doc independently. Cross-encoders take (query, doc) as joint input — much more accurate but 10-100× slower. Cohere Rerank uses this.
Pydantic Settings	How <code>BaseSettings</code> reads from <code>.env</code> . Why <code>Field(default=...)</code> . Type coercion from environment variables.
Background tasks in FastAPI	<code>BackgroundTasks.add_task()</code> schedules after response is sent. Not truly async — same thread pool.
PyMuPDF page extraction	<code>fitz.open()</code> , <code>page.get_text()</code> , why this fails on image-only PDFs.

ADVANCED — For senior roles

Concept	What to Understand
HNSW indexing in ChromaDB	Hierarchical Navigable Small World graphs for approximate nearest neighbor search. ANN trade-off: recall vs. speed vs. memory.
LLM function calling internals	How tool schemas are serialized into the system prompt. How the LLM signals a tool call in its output. Why <code>bind_tools()</code> is different from embedding tool descriptions manually.
Agentic loop reliability	How to prevent infinite loops, detect oscillation, add per-node timeouts.
Production LLM serving	vLLM for throughput, PagedAttention for memory efficiency, batching strategies, how <code>max_new_tokens</code> affects latency.
Vector DB at scale	Qdrant vs. Weaviate vs. Pinecone vs. pgvector. Pre-filter vs. post-filter and their impact on recall.
Evaluation without ground truth	How RAGAS uses LLM-as-judge for some metrics. Why this is a methodological limitation.
Embedding model fine-tuning	Fine-tuning a bi-encoder on domain-specific (query, relevant_passage) pairs using contrastive loss. When this significantly improves retrieval.

10. Final Honest Interview Story

"I built an agentic compliance auditor to solve a specific problem I found interesting: compliance professionals can't trust generic LLMs for regulatory questions because hallucination in legal contexts is harmful — a cited-but-wrong answer is worse than 'I don't know.' The solution had to both retrieve and verify.

The architecture has three layers. The retrieval layer uses ChromaDB with sentence transformer embeddings, combined with keyword term-frequency boosting — the hybrid approach handles both conceptual queries and exact article-number lookups, which pure semantic search missed in early testing. The agent layer is a LangGraph state machine — I chose LangGraph specifically because I needed conditional control over when to reflect and when to loop, which LangChain's simpler AgentExecutor doesn't expose. The evaluation layer uses RAGAS as an offline framework to measure retrieval quality and faithfulness across test queries, which is what drove the switch from fixed-size to section-aware chunking.

The hardest engineering problem was the self-reflection exit condition. My first design had the LLM output structured JSON to signal whether its answer was acceptable. In practice, llama3.2 was inconsistent — it would mark a well-cited answer as needing improvement, causing unnecessary loops. I debugged this by logging all reflection outputs across 30+ test queries and identified the pattern: the LLM was reliable at generating natural-language critique, but unreliable at converting that critique into a binary signal. The fix was to separate those responsibilities: let the LLM generate free-text critique, and use deterministic keyword matching in Python code to make the routing decision. That made behavior consistent and debuggable.

I used Claude as a pair programmer throughout — it helped me generate initial implementations of components I'd already designed, which I then had to understand deeply enough to debug and modify. The agent graph topology, retrieval strategy, and the reflection quality gate were all designed by me and required significant modification from the initial generated code to work correctly. Using AI assistance accelerated development, but it also means you have to work harder to own the understanding — you can't just run the code and claim you built it. Every broken thing I had to fix is where the real learning happened."